SPPU TE BE COMP 👈 Click to join telegram

1. Define Data, Data Object, ADT. Explain with examples.

1. Data

Definition

Data is a collection of **raw facts, figures, or values** that represent information but **do not have meaning on their own** until they are processed.

Explanation

- Data can be **numbers, characters, symbols, images, or sounds**.
- Data is the **input** given to a computer system.
- After processing, data becomes **information**.

Examples

- Numbers: `10`, `25`, `-3`
- Characters: `'A'`, `'z'`
- Strings: `"Shadow"`, `"AIML"`
- Marks of students: `78, 85, 90` • Temperature values: `32°C, 35°C`

👉 Example sentence:

`Marks = 85` → This is data.

`Student passed with 85 marks` → This is information.

---

2. Data Object

Definition

A **Data Object** is a **single instance of data** that has a **specific value** belonging to a particular data type.

Explanation

- Data object represents **one element** from a data set.
- It has a **value** and belongs to a **domain** (set of possible values).
- Data objects are the **basic building blocks** of data.

SPPU TE BE COMP 👈 Click to join telegram

Examples

Data Type	Data Object
Integer	`5`, `-10`
Character	`'A'`, `'@'`
Float	`3.14`, `9.8`
String	`"India"`

👉 Example:

- If data is **marks of students**, then `Marks = 90` is a **data object**.
- If data type is **Integer**, then `25` is a data object of type Integer.

---

3. Abstract Data Type (ADT)

Definition

An **Abstract Data Type (ADT)** is a **logical description of a data type** that defines:

- **What operations can be performed**
- **How data is used** but **not how it is implemented**.

Explanation

- ADT focuses on **behavior**, not implementation.
- It hides internal details (data structure used).
- Implementation can be done using **arrays, linked lists, stacks, etc.**
- ADT follows the concept of **data abstraction**. **General Structure of ADT**

An ADT consists of:

1. **Data objects**
2. **Operations** on those objects

Examples of ADT

1. Stack ADT
Data Objects: Collection of elements **Operations:**

- `push()` – insert element

- `pop()` – remove element
- `peek()` – view top element
- `isEmpty()`

👉 Rule: **LIFO (Last In First Out)**
👉 Implementation can be done using **array or linked list**

---

2. Queue ADT

Data Objects: Collection of elements **Operations:**

- `enqueue()` – insert element
- `dequeue()` – remove element
- `front()`

👉 Rule: **FIFO (First In First Out)**

---

3. List ADT

Operations:

- `insert()` • `delete()`
- `search()`
- `display()`

Real-Life Example of ADT

📱 **Mobile Contact List**

- You can **add**, **delete**, **search** contacts
- You don't know how contacts are stored internally
- This is an example of **ADT**

\

SPPU TE BE COMP Click to join telegram

2. Differentiate between primitive and non-primitive data structures.

Primitive Data Structures	Non-Primitive Data Structures
Primitive data structures store **single, simple values**.	Non-primitive data structures store **multiple values**.
They are **basic data types** provided by the programming language.	They are **derived or complex data types**.
They occupy a **fixed amount of memory**.	They may occupy **variable memory**.
They cannot be divided into smaller data items.	They can be divided into **sub-data structures**.
They store **only one value at a time**.	They can store **a collection of values**.
Operations on them are **simple and fast**.	Operations are **more complex**.
They do not support complex relationships.	They support **linear and non-linear relationships**.
Examples include **int, float, char, boolean**.	Examples include **array, linked list, stack, queue, tree, graph**.

SPPU TE BE COMP Click to join telegram

3. Compare static vs dynamic data structures with examples

Static Data Structures	Dynamic Data Structures
Memory is allocated **at compile time**.	Memory is allocated **at run time**.
Size is **fixed** and cannot be changed.	Size is **flexible** and can grow or shrink.
Memory allocation is done **once**.	Memory allocation and deallocation happen **frequently**.
Memory wastage may occur if size is not fully used.	Memory is used **efficiently**.
Faster access due to **contiguous memory**.	Slightly slower access due to **non-contiguous memory**.
Difficult to insert or delete elements.	Easy to insert or delete elements.
Requires prior knowledge of data size.	Does not require prior knowledge of data size.
Implemented using **arrays**.	Implemented using **pointers / references**.

Example :

1)

1) **Array**

```
int marks[5] = {70, 80, 90, 85, 75};
```

2)

- **Linked List**
- **Stack (dynamic)**

- **Queue (dynamic)**
- **Tree**
- **Graph**

```
struct node {     int
data;     struct
node* next;
};
```

---

4. What are persistent and ephemeral data structures?

---

1. Ephemeral Data Structures Definition

An **ephemeral data structure** is a data structure in which **only the latest version is accessible**, and **previous versions are lost** after modification.

Explanation

- Updates **overwrite** the existing structure.
- Past states **cannot be accessed**.
- Commonly used in traditional programming.
- Efficient in terms of **memory and performance**.

Examples

- Array
- Stack
- Queue
- Linked List

```
int arr[3] = {1, 2, 3};
arr[0] = 10;   // old value (1) is lost
```

👉 Once modified, the old state **no longer exists**.

SPPU TE BE COMP Click to join telegram

2. Persistent Data Structures Definition

A **persistent data structure** preserves **all its previous versions** even after modifications, allowing access to **any version** at any time.

Explanation

- Updates create a **new version** instead of modifying the original.
- Old versions remain **unchanged and accessible**.
- Used in **functional programming**, version control, undo operations.
- Implemented using **structural sharing** to save memory.

Types of Persistence

- **Partial persistence** – old versions can be read, but only the latest can be modified.
- **Full persistence** – all versions can be read and modified.
- **Confluent persistence** – versions can be merged.

Examples

- Persistent Linked List
- Persistent Stack
- Git (version control system)
- Undo/Redo in editors

```
Version 1: [1, 2, 3]
Version 2: [10, 2, 3]
```

---

5. Define an algorithm. Explain characteristics of a good algorithm

---

Algorithm

An **algorithm** is a **finite sequence of well-defined instructions** used to **solve a specific problem** or perform a computation.

SPPU TE BE COMP 👆 Click to join telegram

Formal Definition

An algorithm is a **step-by-step procedure** that takes **input**, processes it through a set of **unambiguous steps**, and produces **output** in a **finite amount of time**.

Example (Simple Algorithm: Find Sum of Two Numbers)

1. Start
2. Read two numbers `a` and `b`
3. Compute `sum = a + b`
4. Display `sum`
5. Stop

Characteristics of a Good Algorithm

A good algorithm must satisfy the following characteristics:

---

1. Input

- An algorithm should accept **zero or more inputs**.
- Inputs must be **clearly specified**.

📌 Example:
Input → two integers `a` and `b`

---

2. Output

- An algorithm must produce **at least one output**.
- Output should be **clearly defined**.

📌 Example:
Output → sum of two numbers

---

3. Definiteness (Unambiguity)

- Each step must be **clear and precise**.
- There should be **no confusion** in the instructions.

❌ Wrong: “Add numbers properly”

☑ Correct: "Add `a` and `b` and store result in `sum`"

---

4. Finiteness

- The algorithm must **terminate after a finite number of steps**.
- Infinite loops make an algorithm invalid.

📌 Example:
A loop that runs from `1 to 10` is finite.

---

5. Effectiveness

- Each step must be **simple**, **basic**, and **executable** in a **reasonable amount of time**.
- Steps should be practically possible.

📌 Example:
Using `a = a + 1` is effective;
"Solve the problem instantly" is not.

---

6. Correctness

- The algorithm should produce the **correct output for all valid inputs**.

📌 Example:
A sorting algorithm must correctly sort **any given list**, not just one test case.

---

7. Efficiency

- A good algorithm should use **minimum time and memory**.
- Efficiency is measured using **time complexity** and **space complexity**.

📌 Example:
Binary search is more efficient than linear search for large data.

---

8. Generality

- The algorithm should solve **all instances of the problem**, not just a specific case.

SPPU TE BE COMP Click to join telegram

📌 Example:
An algorithm that works for **any array size**, not only size = 5.

---

6. Explain algorithm specification with an example.

Algorithm Specification

Definition

Algorithm specification is a **precise and formal description of an algorithm** that explains **what the algorithm does**, **what inputs it accepts**, **what output it produces**, and **the steps involved**, **without depending on any programming language**.

In short:

Algorithm specification tells **how a problem is solved step-by-step** in a **clear and unambiguous manner**.

Purpose of Algorithm Specification

- To **clearly define the problem solution**
- To avoid **ambiguity**
- To help in **correct implementation**
- To analyze **correctness and efficiency**
- To make the algorithm **language independent**

Components of Algorithm Specification

1. **Problem Statement**
2. **Input Specification**
3. **Output Specification**
4. **Algorithm Steps**
5. **Precondition (optional)**
6. **Postcondition (optional)**

SPPU TE BE COMP Click to join telegram

Example: Algorithm Specification to Find the Largest of Two Numbers

Problem Statement

Find the largest of two given numbers.

Input Specification

Two integers: `a` and `b`.

Output Specification

The larger of the two integers.

Precondition

`a` and `b` are valid integers.

Algorithm (Specification)

Algorithm: MaxOfTwo(a, b)

1. Start
2. Read values of `a` and `b`
3. If `a > b`, then Print `a`
4. Else
   Print `b`
5. Stop

Postcondition

The maximum of `a` and `b` is displayed.

SPPU TE BE COMP Click to join telegram

Another Short Example (Sum of Array Elements)

Input

An array `A` of `n` integers.

Output

Sum of all elements of the array.

Algorithm

1. Start
2. Set `sum = 0`
3. For `i = 0` to `n - 1` `sum = sum + A[i]`
4. Print `sum`
5. Stop

---

7. Discuss different algorithm design strategies.

Algorithm Design Strategies

An **algorithm design strategy** is a **general approach or technique** used to develop algorithms for solving problems efficiently.

---

1. Brute Force Strategy

Idea

Try **all possible solutions** and select the correct or optimal one.

Explanation

- Simple and straightforward

SPPU TE BE COMP Click to join telegram

- Does not use optimization
- Often inefficient for large inputs

Examples

- Linear search
- Bubble sort
- Trying all permutations to solve a problem

Pros / Cons

✔ Easy to understand
✘ High time complexity

---

2. Divide and Conquer

Idea

Divide the problem into **smaller subproblems**, solve them independently, and **combine** the results.

Steps

1. Divide
2. Conquer
3. Combine

Examples

- Binary Search
- Merge Sort
- Quick Sort

Advantages

- Efficient
- Reduces time complexity

---

3. Greedy Strategy

Idea

Make the **locally optimal choice** at each step, hoping it leads to a **global optimum**.

SPPU TE BE COMP Click to join telegram

Explanation

- Decisions are final
- No backtracking

Examples

- Prim's Algorithm
- Kruskal's Algorithm
- Dijkstra's shortest path
- Activity Selection Problem

Limitation

- Does not work for all problems

---

4. Dynamic Programming (DP)

Idea

Solve problems by **breaking them into overlapping subproblems** and storing results to avoid recomputation.

Key Properties

- Optimal substructure
- Overlapping subproblems

Examples

- Fibonacci series
- Knapsack problem
- Longest Common Subsequence (LCS)
- Matrix Chain Multiplication

Advantages

- Efficient
- Avoids repeated work

---

5. Backtracking

Idea

SPPU TE BE COMP Click to join telegram

Build solutions **step by step** and **backtrack** when a step leads to no solution.

Explanation

- Systematic search
- Rejects incorrect paths early

Examples

- N-Queens problem
- Sudoku solver
- Maze problems

---

6. Branch and Bound

Idea

Similar to backtracking, but uses **cost bounds** to **prune non-optimal solutions**.

Explanation

- Used for optimization problems
- Eliminates paths that exceed current best solution

Examples

- Travelling Salesman Problem (TSP)
- Job assignment problem

---

7. Randomized Algorithms

Idea

Use **random choices** during execution to improve performance or simplicity.

Explanation

- Result may vary between runs
- Often faster in practice

Examples

- Randomized Quick Sort
- Monte Carlo algorithms

SPPU TE BE COMP Click to join telegram

---

8. Approximation Algorithms

Idea

Produce **near-optimal solutions** when exact solutions are too expensive.

Explanation

- Used for NP-Hard problems
- Guarantees closeness to optimal result

Examples

- Vertex Cover
- Travelling Salesman (approximate)

---

8. Explain time and space complexity with examples

Time and Space Complexity

Complexity

Complexity measures how **efficient** an algorithm is in terms of:

- **Time** (how fast it runs)
- **Space** (how much memory it uses)

---

1. Time Complexity

Definition

Time complexity is the measure of the **amount of time an algorithm takes to execute** as a function of the **input size (n)**.

It tells us **how execution time grows** when input size increases.

SPPU TE BE COMP Click to join telegram

Why Time Complexity is Important

- Compares algorithms **independently of hardware**
- Helps select the **most efficient algorithm**
- Predicts performance for **large inputs**

Big-O Notation

Time complexity is expressed using **Big-O notation**, which represents the **worst-case performance**.

Common notations:

- **O(1)** – Constant time
- **O(n)** – Linear time
- **O(n²)** – Quadratic time
- **O(log n)** – Logarithmic time

Examples of Time Complexity Example

1: O(1) – Constant Time

```
int x = a + b;
```

- Executes once
- Independent of input size

☑ Time complexity: **O(1)**

Example 2: O(n) – Linear Time

```
for(int i = 0; i < n; i++) {
printf("%d", i); }
```

- Loop runs `n` times

☑ Time complexity: **O(n)**

Example 3: O(n²) – Quadratic Time

```
for(int i = 0; i < n; i++) {
for(int j = 0; j < n; j++) {
printf("*");
    }
}
```

- Nested loops

☑ Time complexity: **O(n²)**

Example 4: O(log n) – Logarithmic Time

- **Binary Search**
- Input size reduces by half each step

☑ Time complexity: **O(log n)**

---

2. Space Complexity

Definition

Space complexity is the measure of the **total memory used by an algorithm** as a function of the **input size (n)**.

It includes:

- Input space
- Auxiliary (extra) space
- Temporary variables
- Recursive call stack

Why Space Complexity is Important

- Efficient memory usage
- Important for **large datasets**
- Avoids memory overflow

Examples of Space Complexity

Example 1: O(1) – Constant Space

```
int a, b, sum; sum
= a + b;
```

- Uses fixed number of variables

☑ Space complexity: **O(1)**

Example 2: O(n) – Linear Space

```
int arr[n];
```

- Array size depends on `n`

☑ Space complexity: **O(n)**

Example 3: Recursive Function

```
int fact(int n) {     if(n
== 0)         return 1;
return n * fact(n - 1); }
```

- Uses stack memory for each call

☑ Space complexity: **O(n)**

---

9. Explain Big-O, Ω, Θ notations

Asymptotic Notations

Asymptotic notations describe the **growth rate** of an algorithm's time or space complexity as the **input size (n) becomes large**, ignoring constants and lower-order terms.

SPPU TE BE COMP Click to join telegram

1. Big-O Notation (O)

Definition

Big-O notation represents the **upper bound** of an algorithm's time (or space) complexity.
It describes the **worst-case performance**.
Formally, a function *f(n)* is **O(g(n))** if there exist constants $c > 0$ and n_0 such that **f(n) ≤ c · g(n)** for all **n ≥ n_0**.

Meaning

- "The algorithm will take **at most** this much time." • Guarantees performance will **not exceed** a limit.

Example

```
for(int i = 0; i < n; i++) {
printf("%d", i); }
```

- Runs `n` times

☑ Time Complexity: **O(n)**

Another Example:
Binary Search → **O(log n)** (worst case)

---

2. Big-Omega Notation (Ω)

Definition

Big-Omega notation represents the **lower bound** of an algorithm's time (or space) complexity.
It describes the **best-case performance**.

Formally, *f(n)* is **Ω(g(n))** if there exist constants $c > 0$ and n_0 such that **f(n) ≥ c · g(n)** for all **n ≥ n_0**.

Meaning

- "The algorithm will take **at least** this much time." • Shows the **minimum work required**. **Example**

```
if (arr[0] == key)
return 0;
```

- Key found at first position

SPPU TE BE COMP Click to join telegram

☑ Best case: **Ω(1)**
Another Example:
Linear Search best case → **Ω(1)**

---

3. Big-Theta Notation (Θ)

Definition

Big-Theta notation represents the **tight bound** of an algorithm's complexity. It gives both **upper and lower bounds**.

Formally, $f(n)$ is **Θ(g(n))** if there exist constants $c_1, c_2 > 0$ and n_0 such that $\mathbf{c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)}$ for all $\mathbf{n \geq n_0}$.

Meaning

- "The algorithm grows **exactly** at this rate."
- Most precise asymptotic notation.

Example

```
for(int i = 0; i < n; i++) {
printf("%d", i); }
```

- Always runs `n` times

☑ Time Complexity: **Θ(n)**

---

10. Explain best, average, worst-case complexity

Case Analysis of Algorithm Complexity

Algorithm performance can vary depending on the **input arrangement**. To study this, we analyze algorithms under **three cases**.

SPPU TE BE COMP Click to join telegram

1. Best-Case Complexity

Definition

Best-case complexity is the **minimum time** required by an algorithm for a **specific input** of size `n`.

It represents the **most favorable situation**.

Meaning

- Shows the **fastest possible execution**
- Represented using **Big-Omega (Ω)** notation

Example: Linear Search

- Element is found at the **first position**

```
if (arr[0] == key)
return 0;
```

☑ Best-case time complexity: **Ω(1)**

---

2. Average-Case Complexity

Definition

Average-case complexity is the **expected time** taken by an algorithm over **all possible inputs** of size `n`.

Meaning

- Assumes **all inputs are equally likely**
- Gives a **realistic performance estimate**
- Often difficult to calculate exactly

Example: Linear Search

- Element is found somewhere in the **middle**

```
Average comparisons ≈ n / 2
```

☑ Average-case time complexity: **Θ(n)**

SPPU TE BE COMP Click to join telegram

3. Worst-Case Complexity

Definition

Worst-case complexity is the **maximum time** required by an algorithm for **any input** of size `n`.

It represents the **least favorable situation**.

Meaning

- Guarantees performance **will not exceed** this limit
- Most commonly used in algorithm analysis
- Represented using **Big-O (O)** notation

Example: Linear Search

- Element is at the **last position** or not present

```
for(int i = 0; i < n; i++) {
if(arr[i] == key)
return i; }
```

☑ Worst-case time complexity: **O(n)**

---

11. Step count method for complexity

Step Count Method for Complexity Analysis

Definition

The **step count method** is a technique used to analyze the **time complexity of an algorithm** by **counting the number of basic operations (steps)** executed as a function of the **input size (n)**.

Each **statement execution** is considered **one step**.

SPPU TE BE COMP Click to join telegram

Purpose of Step Count Method

- To **estimate execution time**
- To express time complexity as a **function of n**
- To derive **Big-O, Ω, and Θ** notations

Rules of Step Count Method

1. **Assignment, comparison, arithmetic operation = 1 step**
2. **Loop condition check = 1 step per iteration**
3. **Each statement inside loop = counted separately**
4. **Ignore constants and lower-order terms** for asymptotic analysis
5. Focus on **dominant term**

Example 1: Simple Loop

Algorithm

```
Algorithm SimpleLoop(n)
1.      sum = 0
2.      for i = 1 to n
3.      sum = sum + i
4.      print sum
```

Step Count Analysis

Statement	Number of Executions	Steps
`sum = 0`	1	1
Loop initialization `i = 1`	1	1
Loop condition `i ≤ n`	n + 1	n + 1
`sum = sum + i`	n	n
Increment `i++`	n	n
`print sum`	1	1

Total Steps

$T(n)=1+1+(n+1)+n+n+1T(n) = 1 + 1 + (n + 1) + n + n + 1T(n)=1+1+(n+1)+n+n+1$
$T(n)=3n+4T(n) = 3n + 4T(n)=3n+4$

SPPU TE BE COMP Click to join telegram

Time Complexity

- **Worst Case:** O(n)
- **Best Case:** Ω(n)
- **Average Case:** Θ(n)

Example 2: Nested Loop

Algorithm

```
Algorithm NestedLoop(n)
1.          for i = 1 to n
2.          for j = 1 to n
3.          print "*"
```

Step Count Analysis

Statement	Executions	Steps
Outer loop condition	n + 1	n + 1
Inner loop condition	n(n + 1)	$n^2 + n$
`print "*"`	n^2	n^2

Total Steps

T(n)=n2+n2+n+(n+1)T(n) = n^2 + n^2 + n + (n + 1)T(n)=n2+n2+n+(n+1)
T(n)=2n2+2n+1T(n) = 2n^2 + 2n + 1T(n)=2n2+2n+1

Time Complexity

- **Worst Case:** $O(n^2)$
- **Best Case:** $\Omega(n^2)$
- **Average Case:** $\Theta(n^2)$

---

12. Analyze linear, quadratic, cubic constructs

SPPU TE BE COMP Click to join telegram

1. Linear Construct

Definition

A **linear construct** is a loop that executes **proportional to n**.

General Form

```
for(i = 1; i <= n; i++) {
statement;
}
```

Step Count Analysis

Operation Count Initialization 1
Condition check n + 1
Loop body n
Increment n
T(n)=2n+(n+1)+1T(n) = 2n + (n + 1) + 1T(n)=2n+(n+1)+1 T(n)=3n+2T(n) = 3n + 2T(n)=3n+2

Time Complexity

- **Best Case:** Ω(n)
- **Average Case:** Θ(n)
- **Worst Case:** O(n)

Example

- Linear search
- Printing elements of an array

---

2. Quadratic Construct

Definition

A **quadratic construct** contains **two nested loops**, each running n times. **General**

Form

SPPU TE BE COMP Click to join telegram

```
for(i = 1; i <= n; i++) {
for(j = 1; j <= n; j++) {
statement;
    }
}
```

Step Count Analysis

Operation	Count
Outer loop	n
Inner loop	$n \times n = n^2$
Statement execution	n^2

$T(n)=n2+nT(n) = n^2 + nT(n)=n2+n$

Time Complexity

- **Best Case:** $\Omega(n^2)$
- **Average Case:** $\Theta(n^2)$
- **Worst Case:** $O(n^2)$

Example

- Bubble sort
- Selection sort
- Matrix multiplication (basic)

---

3. Cubic Construct

Definition

A **cubic construct** contains **three nested loops**, each running n times.

General Form

```
for(i = 1; i <= n; i++) {
for(j = 1; j <= n; j++) {
for(k = 1; k <= n; k++) {
statement;
        }
    }
}
```

SPPU TE BE COMP Click to join telegram

Step Count Analysis

Operation	Count
Outer loop	n
Middle loop	n^2
Inner loop	n^3

Operation	Count
Statement execution	n^3

T(n)=n3+n2+nT(n) = n^3 + n^2 + nT(n)=n3+n2+n

Time Complexity

- **Best Case:** $\Omega(n^3)$
- **Average Case:** $\Theta(n^3)$
- **Worst Case:** $O(n^3)$

Example

- Floyd-Warshall algorithm
- Naive matrix multiplication (3 loops)

---

13. Explain linear search.

Linear Search

Definition

Linear search (also called **sequential search**) is a simple searching algorithm that **checks each element one by one** from the beginning of a list until the **required element (key)** is found or the list ends.

SPPU TE BE COMP Click to join telegram

How Linear Search Works

1. Start from the **first element** of the array/list.
2. Compare the current element with the **key**.
3. If it matches, **return the position**.
4. If not, move to the **next element**.
5. Repeat until the element is found or all elements are checked.

Algorithm (Steps)

Algorithm: LinearSearch(A, n, key)

1. Start
2. For `i = 0` to `n - 1`
   If `A[i] == key`, then
   Return `i` (position)
3. If element is not found after the loop
   Return `-1`
4. Stop

Example

Array: `A = [10, 25, 30, 45, 50]` Key to search: `30`

Steps:

- Compare 10 → not equal • Compare 25 → not equal
- Compare 30 → **found**

👉 Element found at **index 2**

Pseudocode / C-like Code

```
int linearSearch(int A[], int n, int key) {
for(int i = 0; i < n; i++) {
if(A[i] == key)             return i;
    }
return -1;
}
```

SPPU TE BE COMP Click to join telegram

Time Complexity

- **Best Case:** Ω(1) — key found at first position
- **Average Case:** Θ(n)
- **Worst Case:** O(n) — key at last position or not present

Space Complexity

- **O(1)** (constant extra space)

Advantages

- Very **simple** to understand and implement
- Works on **unsorted** data
- No extra memory required

Disadvantages

- **Inefficient** for large datasets
- Slower compared to binary search

Applications

- Small datasets
- Searching in **unsorted lists**
- When data size is small or search is infrequent

---

14. Explain binary search.

SPPU TE BE COMP Click to join telegram

Binary Search

Definition

Binary search is an efficient searching algorithm that works on a **sorted array or list**. It repeatedly **divides the search space into two halves** and compares the middle element with the **key** until the element is found or the search space becomes empty.

Precondition

- The list/array **must be sorted** (ascending or descending order).

How Binary Search Works

1. Set `low = 0` and `high = n - 1`
2. Find `mid = (low + high) / 2` 3. Compare `A[mid]` with the key:
   - If equal → element found
   - If key `< A[mid]` → search left half
   - If key `> A[mid]` → search right half
4. Repeat steps until `low > high`

Algorithm (Iterative Binary Search)

Algorithm: BinarySearch(A, n, key)

1. Start
2. Set `low = 0`, `high = n - 1`
3. While `low ≤ high` `mid = (low + high) / 2`

   If `A[mid] == key` Return `mid`

   Else if `key < A[mid]` `high = mid - 1`

   Else `low = mid + 1`
4. Return `-1` (element not found)
5. Stop

SPPU TE BE COMP 👈 Click to join telegram

Example

Array (sorted): `A = [10, 20, 30, 40, 50, 60]`
Key: `40`

Step	low	high	mid	A[mid]
1	0	5	2	30
2	3	5	4	50

Step	low	high	mid	A[mid]
3	3	3	3	40 ✔

👉 Element found at **index 3**

Pseudocode / C-like Code

```
int binarySearch(int A[], int n, int key) {
int low = 0, high = n - 1;      while (low
<= high) {          int mid = (low + high) /
2;          if (A[mid] == key)
return mid;          else if (key < A[mid])
high = mid - 1;          else
             low = mid + 1;
    }
return -1;
                                                                    }
```

Time Complexity

- **Best Case:** Ω(1) — key found at middle
- **Average Case:** Θ(log n)
- **Worst Case:** O(log n)

Space Complexity

- **Iterative version:** O(1)
- **Recursive version:** O(log n) (due to recursion stack)

SPPU TE BE COMP Click to join telegram

Advantages

- Very **fast** for large datasets
- Efficient compared to linear search
- Reduces search space by **half each step**

Disadvantages

- Works **only on sorted data**
- Not suitable for **linked lists** (without extra steps)

Applications

- Searching in sorted arrays
- Databases and file systems
- Dictionary and phonebook searches

---

15. Compare linear vs binary search

Linear Search	Binary Search
Searches elements **sequentially one by one**	Divides the search space into **two halves**
Works on **sorted or unsorted** data	Works **only on sorted** data
Very simple to understand and implement	Comparatively more complex
Checks elements from **start to end**	Checks **middle element first**

SPPU TE BE COMP Click to join telegram

Suitable for **small datasets**	Suitable for **large datasets**
Slower performance	Faster performance
Can be used with **arrays and linked lists**	Best suited for **arrays**
Does not require random access	Requires **random access**

Explain bubble sort.

Bubble Sort

Definition

Bubble sort is a simple **comparison-based sorting algorithm** that repeatedly compares **adjacent elements** and **swaps them** if they are in the wrong order.
With each pass, the **largest element "bubbles up"** to its correct position at the end of the list.

How Bubble Sort Works

1. Start from the **first element** of the array.
2. Compare the **current element** with the **next element**.
3. If the current element is **greater**, **swap** them.
4. Continue this process for the entire array (one **pass**).
5. Repeat passes until **no swaps** are required or all elements are sorted.

Algorithm (Steps)

Algorithm: BubbleSort(A, n)

1. Start
2. For `i = 0` to `n - 2`

SPPU TE BE COMP Click to join telegram

3. For `j = 0` to `n - 2 - i`
4. If `A[j] > A[j + 1]`
5. Swap `A[j]` and `A[j + 1]`
6. Stop

Example

Array: `[5, 1, 4, 2, 8]` **Pass**

1:

- (5,1) → swap → `[1,5,4,2,8]`
- (5,4) → swap → `[1,4,5,2,8]`
- (5,2) → swap → `[1,4,2,5,8]`
- (5,8) → no swap **Pass 2:**

- (1,4) → no swap
- (4,2) → swap → `[1,2,4,5,8]`
- (4,5) → no swap **Pass 3:**

- No swaps → array is sorted

Final Sorted Array: `[1, 2, 4, 5, 8]`

Pseudocode / C-like Code

```
void bubbleSort(int A[], int n) {
for(int i = 0; i < n - 1; i++) {
for(int j = 0; j < n - 1 - i; j++) {
if(A[j] > A[j + 1]) {                    int
temp = A[j];                  A[j] = A[j +
1];
                A[j + 1] = temp;
            }
        }
    }
                                                }
```

Time Complexity

- **Best Case:** Ω(n)
  (When the array is already sorted and optimized version is used)

SPPU TE BE COMP Click to join telegram

- **Average Case:** $\Theta(n^2)$
- **Worst Case:** $O(n^2)$
  (When the array is in reverse order)

Space Complexity

- **O(1)** (In-place sorting, no extra memory)

Characteristics

- Simple and easy to implement
- Stable sorting algorithm
- In-place algorithm
- Inefficient for large datasets

Advantages

- Very easy to understand
- Suitable for **small datasets**
- Useful for educational purposes

Disadvantages

- **Slow** for large datasets
- More comparisons and swaps

Applications

- Teaching sorting concepts
- Sorting very small lists
- When simplicity is more important than efficiency

SPPU TE BE COMP Click to join telegram

17. Explain selection sort.

Selection Sort

Definition

Selection sort is a simple **comparison-based sorting algorithm** that repeatedly selects the **smallest (or largest) element** from the **unsorted part** of the array and places it at the **correct position** in the sorted part.

Basic Idea

- Divide the array into two parts:
  - **Sorted part** (left side)
  - **Unsorted part** (right side)
- In each pass:
  - Find the **minimum element** from the unsorted part ○ **Swap** it with the first element of the unsorted part
- Increase the size of the sorted part by one

Algorithm (Steps)

Algorithm: SelectionSort(A, n)

1. Start
2. For i = 0 to n − 2
3. Set min = i
4. For j = i + 1 to n − 1
5. If A[j] < A[min]
6. Set min = j
7. Swap A[i] and A[min]
8. Stop

SPPU TE BE COMP Click to join telegram

Example

Array: [64, 25, 12, 22, 11]

Pass 1

- Minimum element = 11
- Swap with first element
  → [11, 25, 12, 22, 64]

Pass 2

- Minimum element = 12
- Swap with second element
  → [11, 12, 25, 22, 64]

Pass 3

- Minimum element = 22
- Swap
  → [11, 12, 22, 25, 64]

Pass 4

- Minimum element = 25
- Swap
  → [11, 12, 22, 25, 64]

☑ Array is sorted

Pseudocode / C-like Code void

```
selectionSort(int A[], int n) {
   for(int i = 0; i < n - 1; i++) {
int min = i;        for(int j = i +
1; j < n; j++) {            if(A[j] <
A[min])                min = j;
      }
      int temp = A[i];
A[i] = A[min];
      A[min] = temp;
   }
}
```

SPPU TE BE COMP 👆 Click to join telegram

Time Complexity

- **Best Case:** $\Omega(n^2)$
- **Average Case:** $\Theta(n^2)$
- **Worst Case:** $O(n^2)$

👉 Time complexity is **always n^2**, regardless of input order.

Space Complexity

- **O(1)** (In-place sorting)

Characteristics

- Simple and easy to understand
- In-place sorting algorithm
- Performs **minimum number of swaps**
- Not adaptive (does not check if array is already sorted)
- Not stable (by default)

Advantages

- Easy to implement
- Requires fewer swaps compared to bubble sort
- Useful when **memory writes are costly**

Disadvantages

- Inefficient for large datasets
- Time complexity remains n^2 in all cases

Comparison with Bubble Sort (One Point)

- **Bubble sort** swaps repeatedly
- **Selection sort** swaps **once per pass**

SPPU TE BE COMP Click to join telegram

18. Explain insertion sort.

Insertion Sort

Definition

Insertion sort is a simple **comparison-based sorting algorithm** that builds the **sorted array one element at a time** by inserting each new element into its **correct position** in the already sorted part of the array.

Basic Idea

- The array is divided into:
  - **Sorted part** (left side)
  - **Unsorted part** (right side)
- Take one element from the unsorted part
- Insert it into the correct position in the sorted part
- Repeat until the entire array is sorted

Algorithm (Steps)

Algorithm: InsertionSort(A, n)

1. Start
2. For `i = 1` to `n - 1`
3. Set `key = A[i]`
4. Set `j = i - 1`
5. While `j ≥ 0` and `A[j] > key`
6. Move `A[j]` to `A[j + 1]`
7. Decrement `j`
8. Insert `key` at position `j + 1`
9. Stop

SPPU TE BE COMP Click to join telegram

Example

Array: `[8, 3, 5, 2]`

Pass 1

- Key = `3`
- Insert before `8`
  → `[3, 8, 5, 2]`

Pass 2

- Key = `5`
- Insert between `3` and `8`
  → `[3, 5, 8, 2]`

Pass 3

- Key = `2`
- Insert at beginning
  → `[2, 3, 5, 8]`

☑ Sorted Array

Pseudocode / C-like Code

```
void insertionSort(int A[], int n) {
for(int i = 1; i < n; i++) {
int key = A[i];          int j = i -
1;
        while(j >= 0 && A[j] > key) {
            A[j + 1] = A[j];
j--;
        }
        A[j + 1] = key;
    }
                                                                    }
```

Time Complexity

- **Best Case:** Ω(n)

(Array already sorted) •

Average Case: $\Theta(n^2)$

- **Worst Case:** $O(n^2)$
  (Array sorted in reverse order)

Space Complexity

- **O(1)** (In-place sorting)

Characteristics

- Simple and intuitive
- **Stable sorting algorithm**
- In-place algorithm
- Adaptive (efficient for nearly sorted data)

Advantages

- Efficient for **small datasets**
- Very good for **nearly sorted arrays**
- Simple implementation

Disadvantages

- Inefficient for large datasets
- Slower compared to advanced algorithms like quick sort or merge sort

---

Real-Life Example

- Sorting playing cards in hand
- Arranging books on a shelf one by one

Comparison-Based Sorting	Non-Comparison-Based Sorting
Uses element-to-element comparison	Does not use direct comparisons
Uses $<$, $>$, $\leq$, $\geq$ operators	Uses keys, digits, or positions
General purpose	Data-specific
Minimum time complexity: $\Omega(n \log n)$	Can achieve $O(n)$ time
Less extra memory	Requires extra memory
Slower for very large data	Faster when constraints are known

20. Complexity of recursive algorithms

Complexity of Recursive Algorithms

Introduction

A **recursive algorithm** is one that **calls itself** to solve a problem by breaking it into **smaller subproblems**.
The complexity of recursive algorithms is analyzed in terms of **time complexity** and **space complexity**.

SPPU TE BE COMP Click to join telegram

1. Time Complexity of Recursive Algorithms

Definition

The **time complexity** of a recursive algorithm is the **total number of basic operations executed**, including all recursive calls.

It is usually expressed using a **recurrence relation**.

Recurrence Relation

A recurrence relation expresses the time complexity **T(n)** in terms of smaller inputs.

General form:

T(n)=a T(nb)+f(n)T(n) = a \, T\left(\frac{n}{b}\right) + f(n)T(n)=aT(bn)+f(n)

Where:

- `a` = number of recursive calls
- `n/b` = size of each subproblem
- `f(n)` = work done outside recursion

Example 1: Factorial

```
int fact(int n) {
if(n == 0)
return 1;     return
n * fact(n - 1); }
```

Recurrence relation:

T(n)=T(n−1)+cT(n) = T(n - 1) + cT(n)=T(n−1)+c

Solution:

T(n)=O(n)T(n) = O(n)T(n)=O(n)

Example 2: Fibonacci (Naive Recursive)

```
int fib(int n) {
if(n <= 1)
return n;
    return fib(n - 1) + fib(n - 2); }
```

SPPU TE BE COMP Click to join telegram

Recurrence relation:

T(n)=T(n−1)+T(n−2)+cT(n) = T(n - 1) + T(n - 2) + cT(n)=T(n−1)+T(n−2)+c

Time Complexity:

O(2n)O(2^n)O(2n)

Very inefficient due to **repeated computations**.

Example 3: Binary Search (Recursive)

```
int binarySearch(int A[], int low, int high, int key) {
if(low > high)          return -1;      int mid = (low +
high) / 2;      if(A[mid] == key)          return mid;
else if(key < A[mid])
        return binarySearch(A, low, mid - 1, key);
else
        return binarySearch(A, mid + 1, high, key); }
```

Recurrence relation:

T(n)=T(n/2)+cT(n) = T(n/2) + cT(n)=T(n/2)+c

Time Complexity:

O(log⁡n)O(\log n)O(logn)

---

2. Space Complexity of Recursive Algorithms

Definition

The **space complexity** of a recursive algorithm includes:

- Memory for variables
- **Call stack memory** for recursive calls

Key Point

Each recursive call uses **stack space** until it returns.

Example: Factorial

- Recursive depth = `n`
- Each call uses constant space

Space Complexity=O(n)\text{Space Complexity} = O(n)Space Complexity=O(n)

Example: Binary Search (Recursive)

- Recursive depth = `log n`

Space Complexity=O(log⁡n)\text{Space Complexity} = O(\log n)Space Complexity=O(logn)

---

3. Comparison: Recursive vs Iterative

Aspect	Recursive	Iterative
Time Complexity	Same as iterative (usually)	Same
Space Complexity	Higher (stack space)	Lower
Code simplicity	Simple & elegant	Slightly longer
Risk	Stack overflow	No stack overflow

---

4. Important Points (Exam Ready)

- Time complexity is found using **recurrence relations**
- Space complexity depends on **recursion depth**
- Recursive algorithms may be **inefficient** if overlapping subproblems exist
- Dynamic programming can optimize recursion

NUMERICALS

---

1. Determine time complexity using step-count for nested increasing loop.

SPPU TE BE COMP Click to join telegram

for(i=1; i<=n; i++) for(j=1; j<=i; j++)

sum++;

Given Code

```
for(i = 1; i <= n; i++)      for(j = 1; j <= i; j++)          sum++;
```

Step-Count Analysis

Outer Loop (`i` loop)

- Runs from `i = 1` to `n`
- Number of iterations = **n**

Inner Loop (`j` loop)

- For each value of `i`, `j` runs from `1` to `i`
- Number of executions of `sum++` depends on `i`

Value of `i`	Inner loop runs
1	1 time
2	2 times
3	3 times
…	…
n	n times

Total Step Count for `sum++`

T(n)=1+2+3+…+nT(n) = 1 + 2 + 3 + \dots + nT(n)=1+2+3+…+n

Using formula:

n(n+1)2\frac{n(n+1)}{2}2n(n+1) T(n)=n2+n2T(n) = \frac{n^2 + n}{2}T(n)=2n2+n

---

SPPU TE BE COMP Click to join telegram

Asymptotic Analysis

Ignore constants and lower-order terms:

T(n)=O(n2)T(n) = O(n^2)T(n)=O(n2)

Final Answer

- **Exact step count:**

  T(n)=n(n+1)2T(n) = \frac{n(n+1)}{2}T(n)=2n(n+1)

- **Time Complexity:**
  - **Worst case:** O(n2)O(n^2)O(n2) ∘ **Average case:** Θ(n2)\Theta(n^2)Θ(n2)
  - **Best case:** Ω(n2)\Omega(n^2)Ω(n2)

---

2. Worst, average, best complexity of Bubble Sort for n=8.

---

1. Worst Case (Reverse Sorted Array)

Condition

Array is sorted in **descending order**
Example: `[8,7,6,5,4,3,2,1]` **Number**

of Comparisons

Bubble sort comparisons: n(n−1)2=8×72=28\frac{n(n-1)}{2} = \frac{8 \times

7}{2} = 282n(n−1)=28×7=28

SPPU TE BE COMP Click to join telegram

Number of Swaps

- Every comparison results in a swap
- Swaps = **28**

Complexity

- **Time Complexity:**

  O(n2)⇒O(64)O(n^2) \Rightarrow O(64)O(n2)⇒O(64)

- **Exact operations:** 28 comparisons + 28 swaps

---

2. Average Case (Random Order)

Condition

Elements are in **random order**

Number of Comparisons

- Same as worst case:

8×72=28\frac{8 \times 7}{2} = 2828×7=28

Number of Swaps

- Approximately half of the worst case:

≈14 swaps\approx 14 \text{ swaps}≈14 swaps

Complexity

- **Time Complexity:**

  Θ(n2)⇒Θ(64)\Theta(n^2) \Rightarrow \Theta(64)Θ(n2)⇒Θ(64)

---

3. Best Case (Already Sorted Array – Optimized Bubble Sort)

Condition

Array is already sorted

SPPU TE BE COMP Click to join telegram

Example: `[1,2,3,4,5,6,7,8]`

Comparisons

- One full pass = `n-1 = 7` comparisons
- No swaps → algorithm stops early **Swaps**
- **0 swaps**

Complexity

- **Time Complexity:**

  Ω(n)⇒Ω(8)\Omega(n) \Rightarrow \Omega(8)Ω(n)⇒Ω(8)

⚠ Note: Best case **O(n)** is possible only in **optimized bubble sort** (with swap flag).

---

3. Binary Search trace for element 29 in given sorted array.

Binary Search Trace for Element 29

Given Sorted Array

```
Index : 0   1   2   3   4   5   6   7
Array : 5  12  18  23  29  35  42  50
```

Element to search: `29`

Binary Search Steps (Trace Table)

Step	low	high	mid = (low+high)/2	A[mid]	Result

SPPU TE BE COMP Click to join telegram

1	0	7	3	23	29 > 23 → search right
2	4	7	5	35	29 < 35 → search left
3	4	4	4	29	**Element found**

Result

☑ Element **29 is found at index 4**.

---

4. Trace Insertion Sort for {14, 9, 2, 11, 5}.

Trace of Insertion Sort

Given Array

```
{14, 9, 2, 11, 5}
```

Insertion sort assumes the **first element is already sorted** and inserts each next element into its correct position.

Pass-wise Trace

Initial Array

```
14   9   2   11   5
```

Pass 1 (Insert 9)

- Compare 9 with 14 → shift 14 right
- Insert 9 at position 0

```
9   14   2   11   5
```

Pass 2 (Insert 2)

- Compare 2 with 14 → shift
- Compare 2 with 9 → shift
- Insert 2 at position 0

```
2   9   14   11   5
```

Pass 3 (Insert 11)

- Compare 11 with 14 → shift • Compare 11 with 9 → stop
- Insert 11 at correct position

```
2   9   11   14   5
```

Pass 4 (Insert 5)

- Compare 5 with 14 → shift
- Compare 5 with 11 → shift
- Compare 5 with 9 → shift
- Compare 5 with 2 → stop
- Insert 5

```
2   5   9   11   14
```

Final Sorted Array

```
{2, 5, 9, 11, 14}
```

---

5. Compare Linear vs Binary Search (comparisons) for n=1000.

SPPU TE BE COMP Click to join telegram

Comparison of Linear Search vs Binary Search (n = 1000)

1. Linear Search

Linear search compares the key **sequentially** with each element.

Comparisons:

- **Best case:**
  Element found at first position

  =1 comparison= 1 \text{ comparison}=1 comparison

- **Average case:**
  Element found in the middle

  ≈n2=10002=500 comparisons\approx \frac{n}{2} = \frac{1000}{2} = 500 \text{ comparisons}≈2n=21000=500 comparisons

- **Worst case:**
  Element found at last position or not present

  =n=1000 comparisons= n = 1000 \text{ comparisons}=n=1000 comparisons

---

2. Binary Search

Binary search divides the array into **two halves** each time. (Works only on **sorted array**.)

Comparisons:

- **Best case:**
  Element found at middle

  =1 comparison= 1 \text{ comparison}=1 comparison

- **Average case:**

  ≈log⁡2(1000)≈10 comparisons\approx \log_2(1000) \approx 10 \text{ comparisons}≈log2(1000)≈10 comparisons

- **Worst case:**

  ⌈log⁡2(1000)⌉=10 comparisons\lceil \log_2(1000) \rceil = 10 \text{ comparisons}⌈log2(1000)⌉=10 comparisons

---

SPPU TE BE COMP Click to join telegram

Summary Table (n = 1000)

Case	Linear Search	Binary Search
Best Case	1	1
Average Case	~500	~10
Worst Case	1000	10

---

7. Determine big-O for loop with n(n*n) iterations.*

```
for(i=1; i<=n; i++)

for(j=1; j<=n*n; j++)

        x++;
```

Given Code

```
for(i = 1; i <= n; i++)
for(j = 1; j <= n*n; j++)
x++;
```

Step-wise Analysis

Outer Loop

- Runs from `i = 1` to `n`
- **Number of iterations = n**

Inner Loop

- Runs from `j = 1` to `n * n`
- **Number of iterations = n^2**

SPPU TE BE COMP Click to join telegram

Total Executions of $x++$

Total iterations=n×n2=n3\text{Total iterations} = n \times n^2 = n^3Total iterations=n×n2=n3

Step Count Expression

T(n)=n3T(n) = n^3T(n)=n3

Ignoring constants and lower-order terms.

Big-O Time Complexity

O(n3)\boxed{O(n^3)}O(n3)